

UNIVERSIDAD TECNOLÓGICA NACIONAL

TRABAJO PRÁCTICO INTEGRADOR - PROGRAMACIÓN II

FOOD STORE - SISTEMA DE GESTIÓN DE PEDIDOS

Alumno

Leandro Daniel Schabberger

Profesor

Jerónimo Cortez

Comisión

A25 C2-01

Fecha de entrega

25/06/2026

Descripción

El presente documento corresponde al Trabajo Práctico Integrador de la asignatura Programación II. El proyecto consiste en el desarrollo de una aplicación de consola denominada Food Store, cuyo objetivo principal es gestionar categorías, productos, usuarios y pedidos mediante la aplicación de Programación Orientada a Objetos, colecciones dinámicas, interfaces, excepciones personalizadas y una arquitectura organizada por capas.

ÍNDICE

Introducción_____	3
Marco Teórico_____	3
Objetivos del Proyecto_____	4
Diagrama UML _____	4 a 6
Arquitectura del Sistema_____	6 a 8
Reglas de Negocio, Validaciones y Dificultades Encontradas_____	8 a 10
Conclusiones, Herramientas Utilizadas, Bibliografía y Enlaces_____	10 a 12

1. Introducción

El presente Trabajo Práctico Integrador fue desarrollado para la asignatura Programación II de la Tecnicatura Universitaria en Programación a Distancia de la Universidad Tecnológica Nacional.

El proyecto consiste en el desarrollo de una aplicación de consola denominada Food Store, cuyo propósito es administrar la información de un sistema de pedidos gastronómicos mediante una estructura basada en Programación Orientada a Objetos.

Toda la información se almacena en memoria utilizando colecciones dinámicas y la interacción con el usuario se realiza mediante un sistema de menús implementado en consola.

2. Marco Teórico

Para el desarrollo del sistema se utilizaron diversos conceptos fundamentales de Programación Orientada a Objetos.

Los principales conceptos aplicados fueron:

- Encapsulamiento.
- Herencia.
- Polimorfismo.
- Clases abstractas.
- Interfaces.
- Enumeraciones.
- Colecciones dinámicas.
- Manejo de excepciones.
- Arquitectura por capas.

Asimismo, se respetó el diagrama UML proporcionado por la cátedra para modelar las entidades y las relaciones existentes entre ellas.

3. Objetivos del Proyecto

El objetivo principal del proyecto fue desarrollar una aplicación robusta, modular y organizada que permitiera administrar categorías, productos, usuarios y pedidos mediante una interfaz de consola.

Los objetivos específicos fueron:

- Aplicar Programación Orientada a Objetos.
- Implementar correctamente las relaciones entre entidades.
- Utilizar interfaces para estandarizar comportamientos.
- Implementar excepciones personalizadas.
- Utilizar colecciones dinámicas.
- Aplicar reglas de negocio.
- Desarrollar un sistema estable y fácil de mantener.

4. Diagrama UML

El desarrollo del sistema Food Store se realizó respetando el diagrama UML proporcionado por la cátedra, el cual funcionó como referencia principal durante todo el proceso de diseño e implementación del sistema.

A partir de dicho modelo se construyeron las entidades, sus atributos, sus relaciones y los comportamientos requeridos, manteniendo la coherencia entre el diseño teórico y la implementación práctica.

La estructura final del sistema quedó compuesta por las siguientes entidades:

- Base (clase abstracta).
- Categoría.
- Producto.
- Usuario.
- Pedido.
- DetallePedido.

Asimismo, se implementaron componentes complementarios para representar distintos comportamientos del dominio:

- Interfaz Calculable.
- Enumeración Rol.

- Enumeración Estado.
- Enumeración FormaPago.

Durante la implementación se respetaron las relaciones establecidas por el UML. Las principales relaciones desarrolladas fueron:

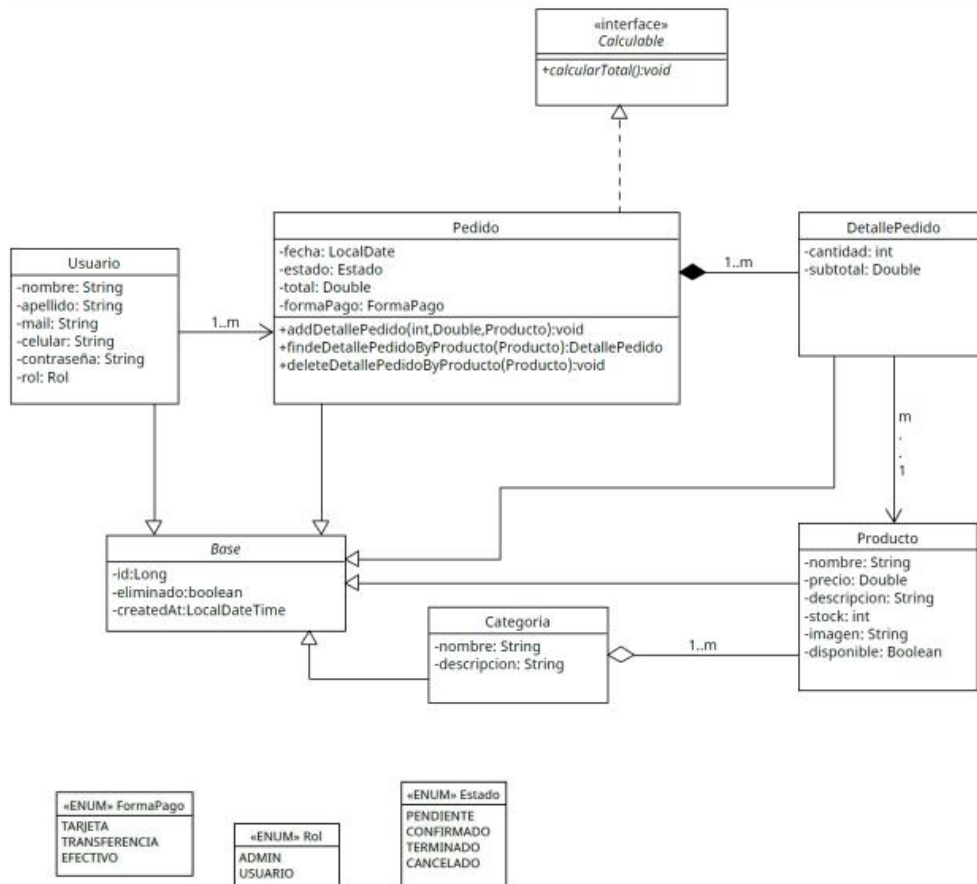
- Una categoría puede contener múltiples productos.
- Un producto pertenece a una única categoría.
- Un usuario puede poseer múltiples pedidos.
- Un pedido está compuesto por múltiples detalles.
- Un detalle de pedido está asociado a un producto.

Además, se implementó una relación de composición entre Pedido y DetallePedido, garantizando que los detalles existan únicamente dentro del contexto de un pedido.

La Figura 1 muestra el diagrama UML utilizado como base para el desarrollo del proyecto.

Figura 1. Diagrama UML del sistema Food Store.

Modelo (UML) y Entidades



5. Arquitectura del Sistema y Decisiones Técnicas

Con el objetivo de mantener una adecuada organización del código y respetar las buenas prácticas de Programación Orientada a Objetos, el proyecto fue desarrollado utilizando una arquitectura por capas.

Cada componente del sistema posee una responsabilidad específica, evitando la mezcla de lógica de negocio, interacción con el usuario y representación de los datos.

La estructura final del proyecto quedó organizada de la siguiente manera:

- Main.java: punto de entrada principal del sistema.
- MenuPrincipal.java: administración del menú principal.
- Menús específicos: interacción con el usuario para cada módulo.
- Services: implementación de las reglas de negocio y validaciones.
- Entities: representación de las entidades del dominio.

- Enums: definición de constantes del sistema.
- Interfaces: definición de comportamientos comunes.
- Exceptions: manejo de errores personalizados.

El paquete entities contiene las entidades principales del dominio y utiliza herencia mediante la clase abstracta Base, la cual centraliza los atributos comunes a todas las entidades.

La clase Base implementa los siguientes atributos:

- id.
- eliminado.
- createdAt.

Esta implementación permitió reducir la duplicación de código y unificar funcionalidades compartidas.

Asimismo, la interfaz Calculable fue implementada exclusivamente por la entidad Pedido, encapsulando la responsabilidad del cálculo del total del pedido mediante el método calcularTotal().

La lógica de negocio fue separada en cuatro servicios principales:

- CategoriaService.
- ProductoService.
- UsuarioService.
- PedidoService.

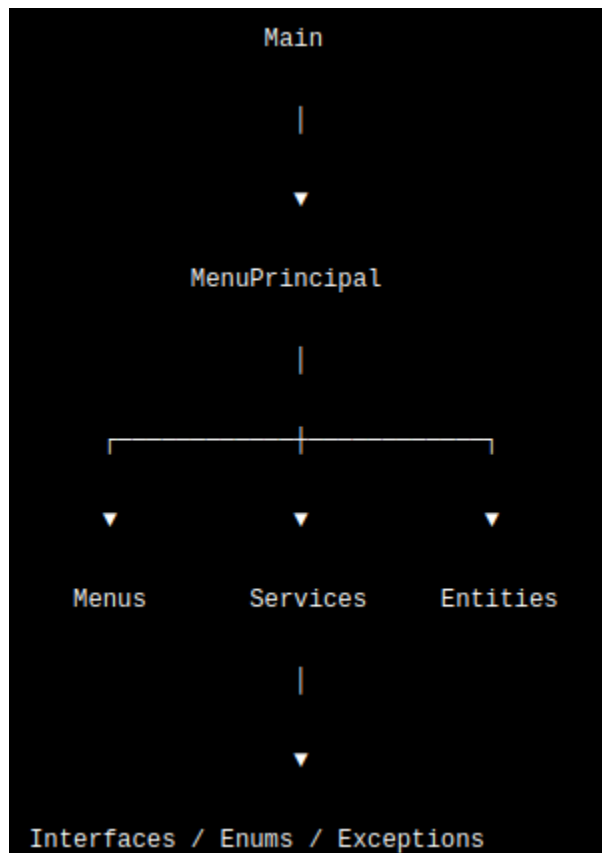
Cada uno de ellos se encarga de gestionar las operaciones CRUD, las validaciones y las reglas de negocio correspondientes.

Por otra parte, la interacción con el usuario se implementó mediante menús independientes:

- MenuCategorias.
- MenuProductos.
- MenuUsuarios.
- MenuPedidos.

Esta separación permitió disminuir el acoplamiento entre componentes y mejorar la mantenibilidad del sistema.

La arquitectura general implementada puede representarse de la siguiente manera:



La implementación de esta arquitectura permitió cumplir con los criterios establecidos por la cátedra, favoreciendo la modularidad, la reutilización y la escalabilidad del código.

6. Reglas de Negocio, Validaciones y Dificultades Encontradas

Durante el desarrollo del sistema se implementaron diversas reglas de negocio con el objetivo de garantizar la integridad de la información y evitar inconsistencias durante la ejecución del programa.

Las principales reglas implementadas fueron las siguientes:

Módulo	Regla implementada
Categorías	No se permiten nombres vacíos.
Categorías	No se permiten categorías duplicadas.
Productos	No se permiten precios negativos.

Productos	No se permiten stocks negativos.
Productos	Todo producto debe pertenecer a una categoría válida.
Usuarios	No se permiten correos electrónicos vacíos.
Usuarios	No se permiten correos electrónicos duplicados.
Pedidos	Todo pedido debe estar asociado a un usuario válido.
Pedidos	La cantidad de productos agregados debe ser mayor a cero.
Eliminaciones	Todas las eliminaciones se realizan mediante baja lógica.

Además, se implementaron diversas validaciones distribuidas en los servicios encargados de la lógica de negocio.

La centralización de estas validaciones permitió evitar que la lógica se dispersara entre los menús y las entidades, respetando la separación de responsabilidades establecida por la arquitectura del proyecto.

Para el manejo de errores se implementaron excepciones personalizadas, permitiendo controlar situaciones específicas del dominio del problema.

Las excepciones desarrolladas fueron:

- `DatoInvalidoException`.
- `EntidadNoEncontradaException`.
- `MailDuplicadoException`.

Asimismo, se incorporaron confirmaciones adicionales en las operaciones de eliminación mediante respuestas de tipo S/N, reduciendo la posibilidad de eliminar información de forma accidental.

Durante el desarrollo del proyecto surgieron diversas dificultades técnicas que debieron resolverse.

Las principales fueron las siguientes:

Dificultad	Solución implementada
------------	-----------------------

Modelar correctamente las relaciones entre entidades	Se implementaron asociaciones y composiciones respetando el UML proporcionado por la cátedra.
Evitar la duplicación de datos	Se agregaron validaciones específicas en los servicios.
Mantener la integridad de la información	Se implementaron reglas de negocio centralizadas.
Separar responsabilidades	Se desarrolló una arquitectura organizada por capas.
Manejar errores durante la ejecución	Se implementaron excepciones personalizadas y mensajes informativos.
Evitar la eliminación definitiva de los objetos	Se implementó la baja lógica mediante el atributo eliminado.

Estas decisiones permitieron construir una aplicación más robusta, modular y fácil de mantener, respetando tanto los requerimientos funcionales como las buenas prácticas de Programación Orientada a Objetos.

7. Conclusiones, Herramientas Utilizadas, Bibliografía y Enlaces

El desarrollo del presente Trabajo Práctico Integrador permitió consolidar y aplicar los conocimientos adquiridos durante la asignatura Programación II.

A lo largo del proyecto fue posible implementar los principios fundamentales de Programación Orientada a Objetos mediante una aplicación funcional y organizada, respetando tanto los requerimientos técnicos establecidos por la cátedra como las buenas prácticas de desarrollo de software.

Uno de los principales aprendizajes obtenidos fue la importancia de la separación de responsabilidades dentro de una aplicación. La implementación de una arquitectura por capas permitió dividir claramente la interacción con el usuario, la lógica de negocio y la representación de los datos, favoreciendo la reutilización y el mantenimiento del código.

Asimismo, la utilización de una clase abstracta, interfaces, enumeraciones, excepciones personalizadas y colecciones dinámicas permitió desarrollar una solución modular y escalable.

Otro aspecto importante fue la implementación de validaciones y reglas de negocio, garantizando la integridad de la información y evitando inconsistencias durante la ejecución del sistema.

Durante el desarrollo también se fortalecieron habilidades relacionadas con el análisis de requerimientos, la interpretación de diagramas UML y la resolución de problemas mediante soluciones estructuradas y reutilizables.

El resultado final fue una aplicación de consola estable, organizada y alineada con los objetivos académicos propuestos para la asignatura.

Herramientas utilizadas

Las herramientas empleadas durante el desarrollo del proyecto fueron las siguientes:

- Java 21.
- Apache NetBeans.
- Git.
- GitHub.
- Microsoft Word.
- Material teórico proporcionado por la cátedra.

Pruebas realizadas

Para verificar el correcto funcionamiento del sistema se realizaron pruebas manuales sobre todas las operaciones implementadas.

Entre ellas se destacan:

- Creación de categorías.
- Modificación de categorías.
- Eliminación lógica de categorías.
- Creación de productos.
- Modificación de productos.
- Eliminación lógica de productos.
- Creación de usuarios.
- Modificación de usuarios.
- Eliminación lógica de usuarios.
- Creación de pedidos.
- Agregado de productos a un pedido.
- Eliminación de productos de un pedido.
- Actualización del estado de un pedido.
- Actualización de la forma de pago de un pedido.

- Eliminación lógica de pedidos.
- Pruebas de validación de errores y excepciones.

Bibliografía y Webgrafía

- Oracle Java Documentation.
- Oracle Java Collections Framework.
- Oracle Java Exception Handling.
- Oracle LocalDate Documentation.
- Documentación oficial de Java.
- Material proporcionado por la cátedra.

Enlaces del Proyecto

Repositorio GitHub

<https://github.com/LSchabberger/UTN-TUPaDProgramacion2/tree/main/TPI%20Programacion%20II>

Video demostrativo

<https://www.youtube.com/watch?v=1pHz62Sx5il>